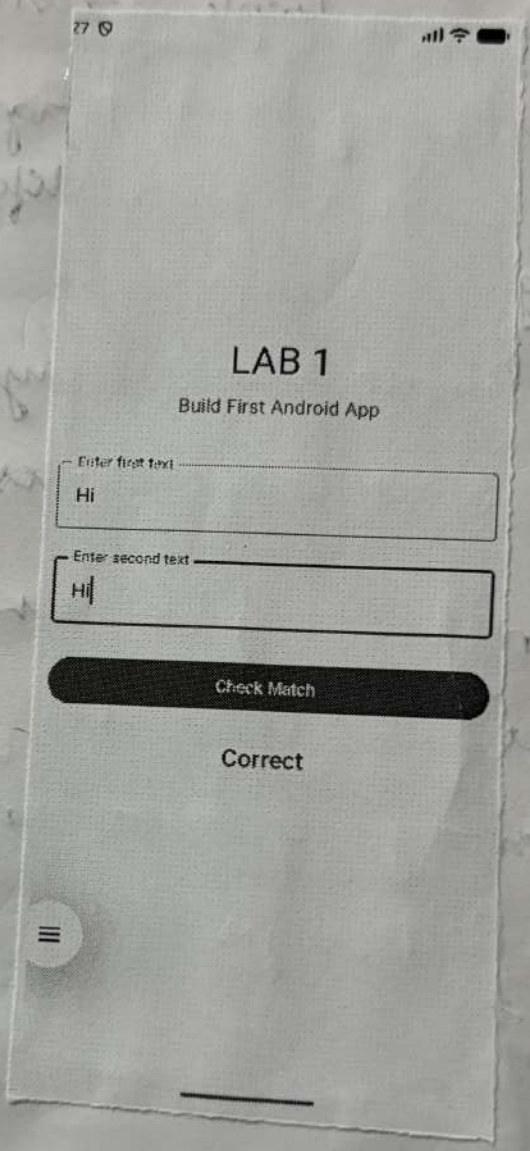
S.no	Date	Title	Teacher's Sign
1.		Drawing Shapes on canva using custom view in Android	BS
2.		Build first Android app - text match validation using jetpack compose	BS
3.		App navigation - activity transition between multiple screens in Android	BS
4.		The activity and Fragment lifecycles - Simple timer with lifecycle event loggers.	BS
5.		App architecture (UI layer)	BS
6.		App architecture (Persistance)	BS
7.		Advanced recycles view use cases	BS
8.		connect to internet	BS
9.		Repository pattern and work manager	BS
10.		App - UI Design	

Completed
BS


LAB 1
Build First Android App
Enter first text
Hi
Enter second text
Hi
Check Match
Correct

Ex no: 1
Date:

Drawing Shapes on canva using custom view in Android

Aim:

To create an android application demonstrating custom drawing using canvas with interactive shape controls to understands how to render 2D graphics programmatically in Android.

Algorithm:-

1. Open Android Studio and create a new android project with an empty activity template.
2. Name the project and set the package name as com.example.lab1.
3. Create a new Kotlin class that extends view and override the onDraw() method.
4. Inside onDraw(), create a paint object and set properties like colour, stroke width and style.
5. Use the canvas object passed to onDraw() to draw shapes such as lines and circles.
6. In the main layout XML file, add the custom view along the with four buttons labeled lines, circle, show all, and clear all.
7. In Main activity, get references to all buttons and the custom view using fen viewById.
8. Maintain boolean flags to track wha shapes should be visible.

9. Set on click listener for ... to update the flags accordingly.
10. Call envalidate() on the custom view after each button click to trigger a redraw
11. In the onDraw() method, check the flags and draw only the shapes that are currently active.
12. Run the application on an emulator or physical device and verify each button works correctly.

Output:-

The application displays a white canvas area at the top of the screen. A horizontal blue lene and a red circle are drawn on the canvas. Below the canvas, four buttons are displayed. Pressing lene shows only the lene. Pressing circle shows only the circle. Press shows all displays both shapes simultaneously. Pressing clear all removes all shapes from the canvas. The title of the app shows experement 2 and layout is displayed.

Result:-

The Android application was successfully created and executed. The custom view correctly renders geometric shapes using canvas and paint. The button interactions work as expected, toggling the visibility of shapes dynamically. The envalidate() method successfully triggers view redraws on each user interaction

Ex no: 2
Date:

Aim:-
To
jetpack
from
entered
the
function
Algorith
1.
Android
templat
2.
as
3.
content
4.
UI for
5
App
state
of
store
encr
elemen

Ex no: 2
Date:

Build First Android-App - Text match validation using Jetpack compose.

Aim:-

To build an android application using jetpack compose that accepts two text input from the user and validates whether the entered strings match each other, demonstrating the use of state management and composable functions.

Algorithm:.

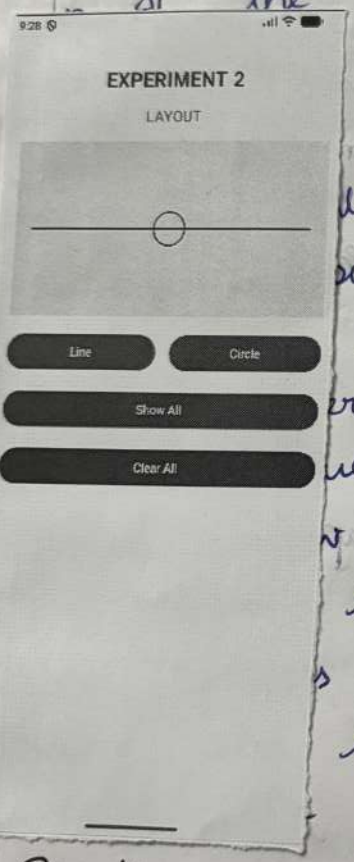
1. Open Android studio and create a new Android project with the empty compose activity template.
2. Name the project and set the package name as com. example. lab2.
3. Open MainActivity.kt and set up the compose content inside the set content block.
4. Apply the LAB2 Theme to wrap the entire UI for consistent theming.
5. Create a composable function named validate App that holds the UI logic.
6. Inside validation App, declare two mutable state variable using remember and mutable Stat of to store the values of the two text fields.
7. Declare another mutable state variable to store the results message such as correct or encorrect.
8. Use a column composable to arrange elements vertically with proper padding and alignment.

9. Add a text composable at the top displa
LAB 1 as the title and budd first android
app as the subtitle.

10. Add two outlined Text feld composable with
labels enters first text and enter second
text, bendeng them to the respective stat
variables.

11. Add a Button composable labeled check math

12. Inside the button's click lambda, compare th
two State variable useng equalety check.
If the values are equal, set the result s
otherwise set it to encorrect.



licaticon deplays a screen titled LAB
subtitle budd first android capp.
title are two outlined text fel
er first text and enter second text
ue button labeled check math es
y the fields. When the user enters
text such as hhh en both fields
s the button, the text correct appe
button. When different texts are
result changes to encorrect.

Result:.

The android application useng jetpack compose was successfully created and executed on emulator State management useng remember and mutable State of worked correctly. The strong comparison log functioned as expected and the UI updated reactively based on user enput. The composable structure and themeng were applied without errors

Ex no:
Date:

Aim:.

To de
an andro
direcional
Android a
concept of

Algorithm:.

1. Ope
Android pr
2. Na
name as
3. Op
Screen la
4. A
displayeng
5. Ad
descriptive
above to
6. cr
activity.kt
compact Ac
7. D
title se
Seond
8.
descriptive
9.
or oncl

Ex no:
Date:

App Navigation - Activity Transition between Multiple Screens on Android

Aim:

To demonstrate multi-screen navigation in an android application by implementing bi-directional transitions between two activities using Android intents, helping to understand the concept of activity stack and navigation flow.

Algorithm:

1. Open Android Studio and create a new Android project with empty activity template.
2. Name the project and set the package name as com.example.lab3.
3. Open activity_main.xml and design the main screen layout using jetpack compose or XML
4. Add a title text composable or textview displaying LAB 3 and a subtitle App Navigation.
5. Add a Bug labeled go to second page & a descriptive text below it saying click the button above to navigate to another page.
6. create a new Kotlin file named second activity.kt extending componentActivity or app compact Activity.
7. Design the second screen layout with a title second page and a subtitle welcome to the second page.
8. Add a button labeled go back to main & a descriptive text below it.
9. In main activity, set an onclick listener or onclick lambda on the first button

10. Inside the click handler, create an intent object with the current context and second activity class as the target.

11. Call startActivity() passing the intent to navigate to second activity.

In Second Activity, Set an OnClick listener, on to main button.

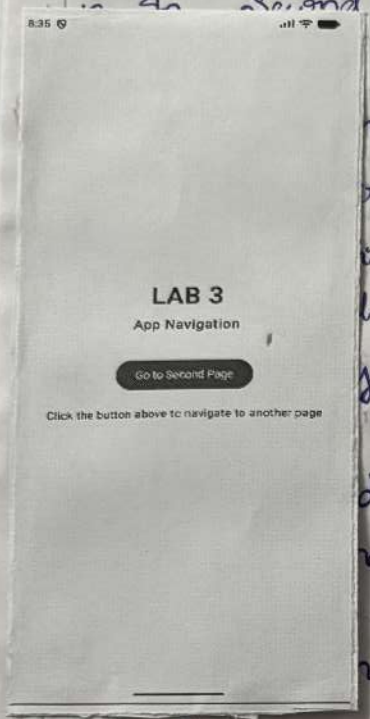
() onside the click handler to pop ty off the back stack and return ivity.

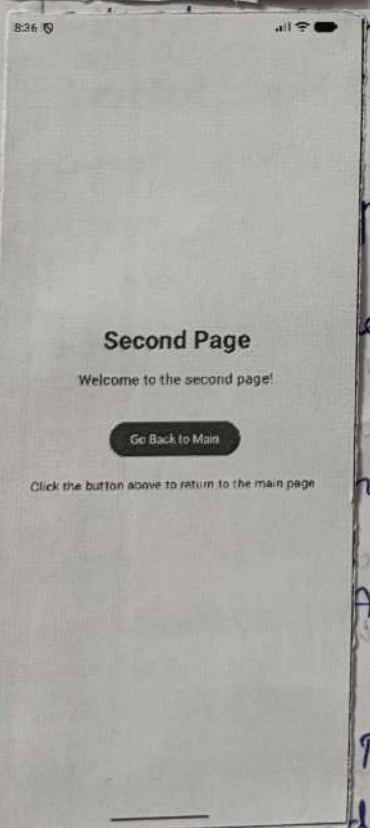
droid Manifest.xml and verify that both and second activity are registered application tag.

d run the application on the d test navigation in both direction

screen displays the title LAB 3 and navigation. A dark blue button. to second page is shown in the a description text below it. When th pressed, the app transitions to the which displays the title Second page e welcome to the second page.

ndroid application was successfully creat activities. Navigation from Main Activity Activity using intent worked correctly. navigation using finish() also worked Both activities were correctly registered manifest.xml. The Activity back Stack was maintained properly throughout the navigation flow.





Ex no: 4
Date:

Aim:

To

fragment
applicaton
event
systems
enteracte

Algorithm

1.

android
_te

2.

package

3.

layout

4.

the
fragme

to

value
to

labeled

events

The application displays a well structured user interface with various UI components such as text view, Edit text & buttons

Thus, the android application demonstrating app UI design was successfully built and executed

Exno: 10

App - UI Design

Aim:-

To design and implement an attractive and user-friendly user interface (UI) for an android application using layout components and UI widgets.

Algorithm:

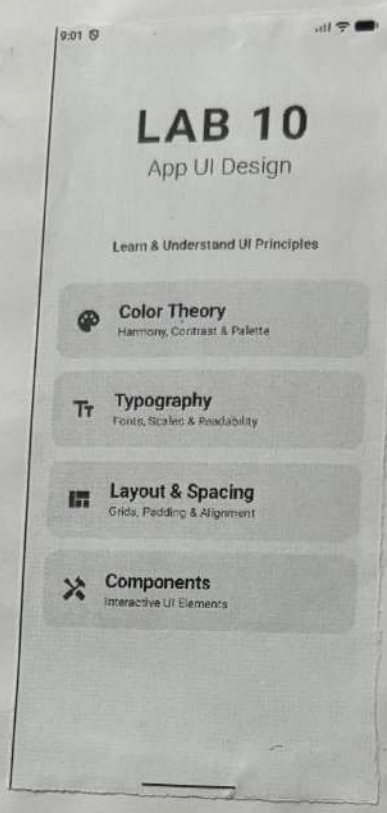
1. Open Android Studio and create a new Android project using empty activity
2. Name the project and set the package name as com.ex.lab00.
3. Open the layout file activity_main.xml
4. Choose a suitable layout structure such as linear layout or constraint layout.
5. Add textview to display the title or labels on app
6. Add Edit text fields to allow the users to enter or input.
7. Add Button to perform specific actions
8. Apply styles, colors and margins to improve the UI appearance
9. Arrange the UI component properly for a clean & responsive design
10. Save the layout and connect the UI element

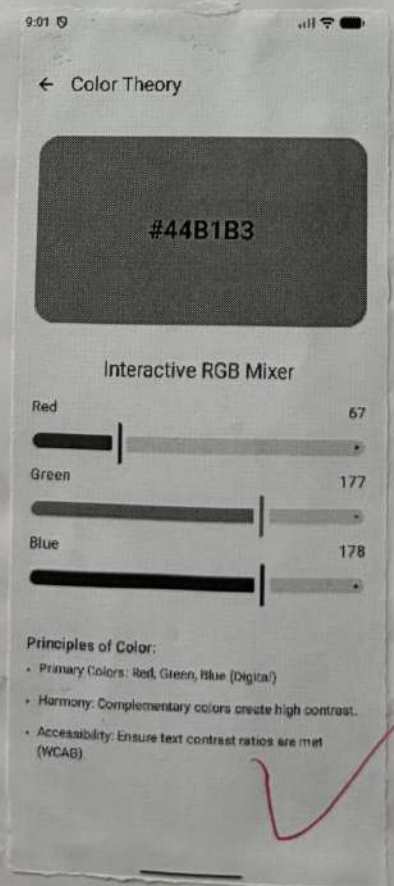
Exno: 10

Aim:-

To
user-
applica
UI

Algorc
1. Op
Android
2. N
name
3. Op
4. C
lorea
5. A
lab
6. Ac
to
7.
8. A
the
9.
for
10.
UI





10. Run the application and observe the background task execution.

Output:

The application demonstrate data management using the repository pattern and background task execution using workmanager

Result:

Thus the android application emplementing the repository pattern executed successfully

Ex no: 9 Reposotory pattern and work manager

Aim:

To understand the reposotory pattern and workmanager on android app archtectaore for managerg data operatcons and performerg background tasks efficcently.

Algorothm:

1. Create a new Android project and set the package name as com.ex.lab9.
2. Add the workManager dependancy en the build.grade.
3. Create a reposotory class to manage data operatcon between the data source and UI layer
4. Implement functcons en the reposotory to fet or store data.
5. Create a worker class that extends worker to perform background tasks.
6. Override the doWorks() method to defene th task that should vun en the background.
7. Snetialeze workHanger es the HaenActevety.
8. Create a workrequest to schedules the backgrou tasks
9. Use the reposotory class to provide data

Ex no: 9

Aim:

To

work ma

for m

backgr

Algorith

1. Crea

packag

2. Ad

build

3. Crea

operat

layer

4. Imp

or

5. Cre

perfor

6. Over

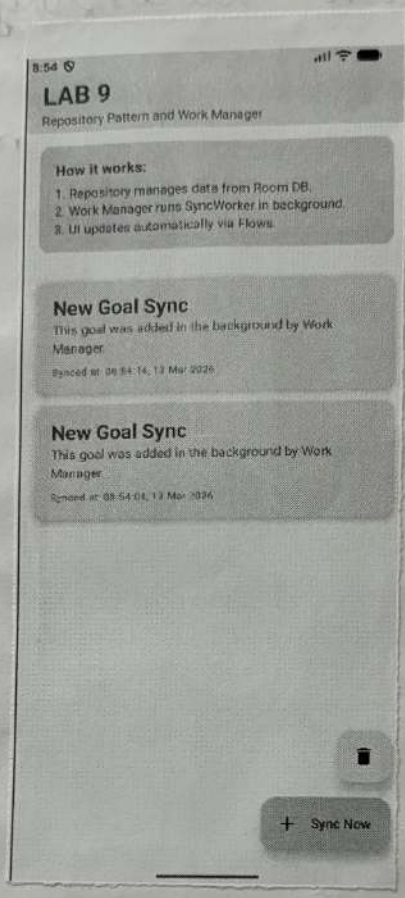
task

7. Ins

8. Cre

tasks

9. U



10. Budd and run the application on the Android emulator or device with an active internet connection.

Output:

The application successfully connects to the internet and retrieves data from a web service, displaying the information on the screen.

Result:

Thus, the Android application demonstrating internet connectivity was successfully built and executed.

Ex no: 8 Connect to the Internet.

Aim:

To understand how to connect an Android applications to the internet and retrieves data from an online source using network connectivity.

Algorithm:

1. Name the project and set the package name as com.ex.labs
2. Open the file Android Manifest.xml.
3. Add the internet permission using: uses-permission. android: name="and wood. permission. INTERNET'.
4. Add a textview to display the data retrieved from the internet.
5. Add a button to initialise the internet request.
6. In MainActivity.java, declare variables for UI components.
7. Use a network library or HTTP connection to request data from web URL.
8. Write code inside the button click listener to connect to the internet and fetch data.
9. Display the retrieved data in textview.

Ex no: 9

Aim:

To

work ma

for m

backgr

Algorith

1. Crea

packag

2. Ad

build

3. Crea

operat

layer

4. Imp

or

5. Cre

perfor

6. Over

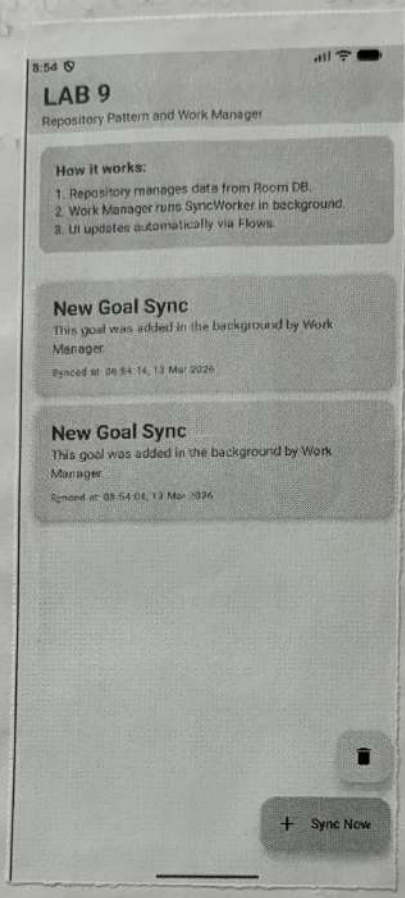
task

7. Ins

8. Cre

tasks

9. U



10. Budd and run the application on the Android emulator or device with an active internet connection.

Output:

The application successfully connects to the internet and retrieves data from a web service, displaying the information on the screen.

Result:

Thus, the Android application demonstrating internet connectivity was successfully built and executed.

Ex no: 8 Connect to the Internet.

Aim:

To understand how to connect an Android applications to the internet and retrieves data from an online source using network connectivity.

Algorithm:

1. Name the project and set the package name as com.ex.labs
2. Open the file Android Manifest.xml.
3. Add the internet permission using: uses-permission. android: name="and wood. permission. INTERNET'.
4. Add a textview to display the data retrieved from the internet.
5. Add a button to initialise the internet request.
6. In MainActivity.java, declare variables for UI components.
7. Use a network library or HTTP connection to request data from web URL.
8. Write code inside the button click listener to connect to the internet and fetch data.
9. Display the retrieved data in textview.

Ex no: 8

Aim:

To

applicat

data

connecte

Algorith

1. Na

name

2. Op

3. A

permis

INTERN

4. A

retrieve

5. A

reque

6. I

compon

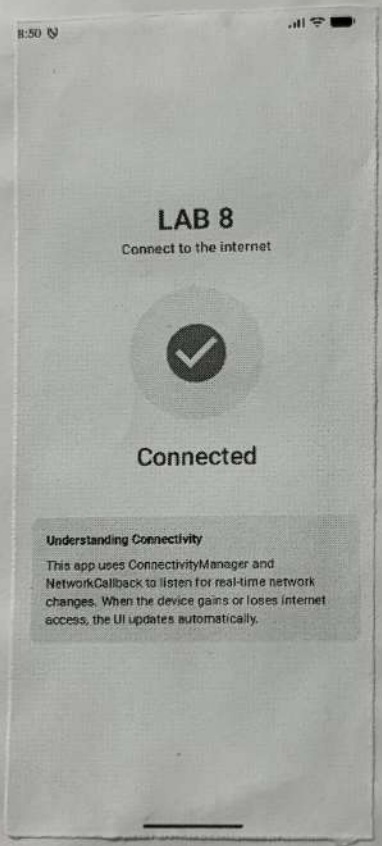
7. U

to r

8.

to

9.



listeners, dynamic data updates and initialize with layout manager and adapter in main activity.

10. Build and run the application

Output:-

The applications displays a scrollable list of items recyclerview, allowing efficient data display and interaction with list elements.

Result:-

Thus, the Android application demonstrating advanced, recyclerview use-cases was successfully built and executed.

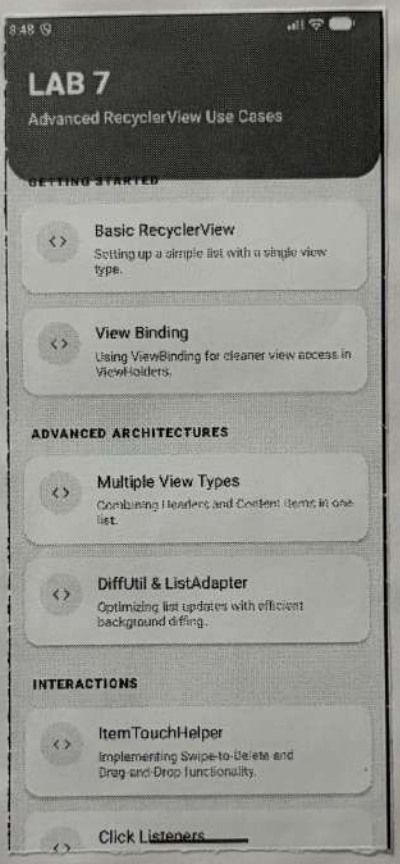
Ex no: 7 Advanced recycles view use cases

Aim:.

To understand the advanced usage of recycle view on android for efficiently displaying large sets of data with features such as custom layouts.

Algorithm:.

1. Create and Name the project and set the package name as com. ex. lab7.
2. Add the recyclerviews dependancy en build gradle file.
3. Open activity-main.xml and add a recycler view.
4. Create a layout file to design each item displayed en the recycler view.
5. Create a model class to store the data for each item.
6. Create a adapter class that extends recycler view adapter.
7. Implement viewholder to hold references to item views.
8. Bind the data from model class to the viewers inside the onBind ViewHolder() method
9. Implement advanced features like clock



Ex no

Aim:-

I

vew

larg

cust

Algo

1-

the

2.

gra

3.

vie

4.

des

5.

fo

6

vc

10. Budd and run the application on the android emulator or device.

Output:

The application displays the title LAB-6 app architecture (persistance) and allows the user to store and retrieve data using a persistence mechanism.

Result:

Thus, the android application demonstrating the persistance layer in App architecture was successfully built and executed.

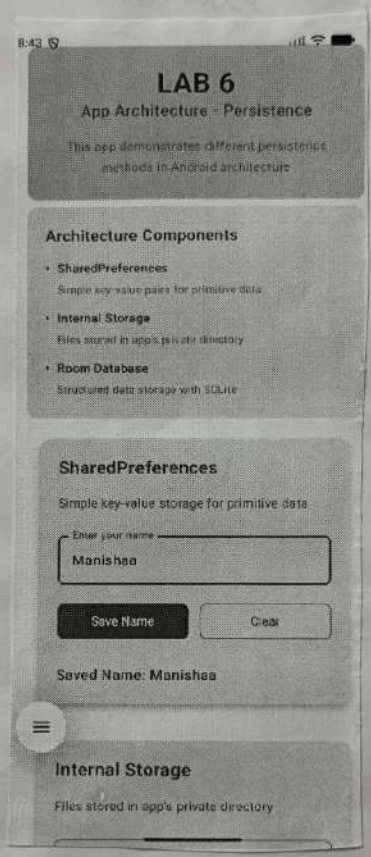
Ex no: 6 App Architecture (Persistence)

Aim:

To understand the persistance layer en android app architecture, which is used to store and retrive data locally using databases or storage mechanisms so that the data remains available even after the app is closed.

Algorithm:

1. Open Android Studio and create a new android project using empty activity.
2. Name the project and set the package name as com.ex.lab.
3. Open the layout file activity_main.xml.
4. Add textview to display the title of the application.
5. Add Edit Text fields to enter the data that needs to be stored.
6. Add buttons such as save and retrive.
7. Create variables en MainActivity.java for UI components.
8. Implements a local persistance mechanism such as Shared preferences or SQLite database to store the data.
9. Write code en the save button click listen and retriever button click listener to fetch & display the data.


LAB 6
App Architecture - Persistence
This app demonstrates different persistence methods in Android architecture
Architecture Components
SharedPreferences
Simple key-value pairs for primitive data
Internal Storage
Files stored in app's private directory
Room Database
Structured data storage with SQLite
SharedPreferences
Simple key-value storage for primitive data
Enter your name
Manishaa
Save Name
Clear
Saved Name: Manishaa
Internal Storage
Files stored in app's private directory

10. Run the application on the android emulator or physical device and observe how UI layer interacts with user inputs and updates the screen.

Output:

The application displays the title LABS - APP architecture (UI layers) and provides UI components such as textview, edit text and buttons.

Result:

Thus, the Android application demonstrating the UI layer on App architecture was successfully built and executed.

Ex no: 5

Date:

App architecture (UI layer)

Aim:

To understand the UI layer in Android app architecture which is responsible for displaying data on the screen and handling user interaction using activities, fragments and view components

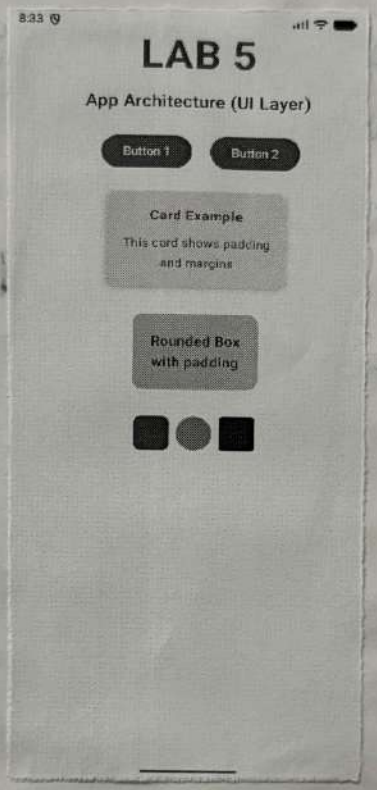
Algorithm:

1. Open Android Studio and create new project using empty activity.
2. Name the project and set the package name as com.ex.lot5
3. Open the layout the activity_main.xml.
4. Add a textview to display the application title.
5. Add edit text field to accept user output.
6. Add buttons to perform actions such as submit or display data
7. create variables in MainActivity.java to ref UI components.
8. Implement onclick listeners for the button to handle user interaction.
9. Update the textview based on user input to demonstrate.

Ex no: 5

Date :

Aim:



Output:-

The application displays the title LAB4 and subtitle Activity and Fragment lifecycles at the top. A Simple Timer section shows the elapsed time in the format 00:00:02.49. Three button START, PAUSE and STOP are displayed in a row. Below the buttons, an activity lifecycle events section shows a scrollable list of time stamped log entries recording each lifecycle event. In the logcat panel at the bottom of Android Studio, log entries confirms events such as timer started and timer paused with their respective timestamps.

Result:-

The Android application was successfully built and executed. The timer functionality using Handl and runnable worked correctly with START, PAUSE and STOP. All six activity lifecycle methods were successfully overriden and their events were logged both on logcat using log.d() and displa on the screen in real time. The lifecycle eve sequence was observed as onCreate, onStart and onResume on launch, and onPause and onStop when the app was backgrounded, confirming correct lifecycle behavior.

Ex no: 4
Date:

The Activity and Fragment lifecycles - Simple Timer with lifecycle event logging.

Aim:

To understand the Android activity and fragment lifecycle by building a simple timer application that logs each lifecycle call back event with timestamps, demonstrating how the systems manages activity states during user interactions.

Algorithm:-

1. Open Android Studio and create a new android project using the empty activity template with XML layouts
2. Name the projects and set the package name as com.example.lab4
3. Open activity_main.xml and design the UI layout using linear layout or ConstraintLayout
4. Add a textview at the top to display the app title lab4 and subtitle activity and fragments lifecycles.
5. Add another TextView labeled Simple Timer to serve as the timer section header.
6. Add a Textview to display the timer value in the format HH:MM:SS:ms initialized to 00:00:00:00
7. Add three buttons in a horizontal row labeled START, PAUSE & STOP.
8. Add a TextView labeled Activity lifecycle events as a section header below the buttons.

9. Add a ScrollView containing a TextView to display the lifecycle log entries.

10. In MainActivity.kt, declare a Handler and a Runnable to update the time every millisecond

11. Declare variable to track elapsed time, running state, and start time.

12. In the Start button, click listener, record the start time and post the Runnable to the Handler.

13. In the Pause button click listener, remove the runnable from the handler and save elapsed time.

14. In the Stop button click listener, stop the timer and reset the elapsed time to zero

15. Override all lifecycle methods including onCreate, onStart, onResume, onPause, onStop and onDestroy

16. Inside each overriden method, call Log.d() with a tag and a message containing the current timestamp.

17. Also append the lifecycle event message to the textview inside the ScrollView so it is visible on screen

18. Build and run the app on the emulator interact with the timer buttons and observe the lifecycles logs both on screen and on Logcat.

Output:

The
and
the
elapsed
Three
on a
events
stamp
In th
Studio
timer
respec

Result
T
and
and
and
succe
logg
on
sequ
and
on